

Resilience & Circuit Breakers

How to build microservices that survive failure gracefully

Circuit Breaker

Retry

Fallback

Bulkhead

Polly

● CLOSED

● OPEN

● HALF-OPEN



What We'll Cover

01

The Problem

Cascading failures — how one broken service kills everything

02

What is Resilience?

Handling failure gracefully instead of crashing

03

The Circuit Breaker

The pattern, 3 states, and how it works

04

Retry & Fallback

Retry with backoff, fallback responses, timeout

05

Bulkhead Pattern

Isolate failures so one service can't drain everything

06

Polly in .NET

Hands-on implementation with real C# examples

Cascading Failures

One broken service can bring down the entire system.



✗ Without Circuit Breaker

- OrderService keeps calling PaymentService — 0% succeed
- Every call waits for the timeout (e.g. 10 seconds)
- Threads pile up waiting → thread pool exhausted
- OrderService also goes down → now Inventory fails too
- One bad service = entire platform offline

✓ With Circuit Breaker

- Circuit detects repeated failures and OPENS
- Subsequent calls INSTANTLY return an error/fallback
- No waiting, no thread blocking
- PaymentService gets time to recover
- OrderService stays healthy and serves other requests

The 3 States

The circuit breaker is a state machine — it switches between 3 states based on what it observes.

❓ CLOSED

Normal Operation

Everything is working. Requests flow through normally. The breaker silently counts failures in the background.

If failures stay below the threshold → stays CLOSED forever

● OPEN

Circuit Tripped

Too many failures detected. ALL requests instantly fail without calling the service. No waiting, no timeouts.

After a configured wait (e.g. 30s) → moves to HALF-OPEN

❓ HALF-OPEN

Testing Recovery

Allows a small number of test requests through to check if the service has recovered.

Success → CLOSED | Failure → OPEN again

failures > threshold →

timeout expires →

← success: back to CLOSED

Retry, Fallback & Timeout

Retry Pattern

Automatically re-attempt a failed operation a limited number of times before giving up.

Use for: transient, one-off failures (network blip, brief timeout)

⚠ Don't retry on 4xx errors — only on 5xx/timeouts

Fallback Pattern

When all retries fail, return a safe default response instead of an error.

Options: cached data · default value · empty list · degraded response

Goal: user sees something useful, not a 500 error

Timeout Pattern

Give up if a call takes too long. Don't wait forever for a hung service.

Without timeout: threads block for 60s+ → system slows to halt

How They Work Together

1. Request fails
2. Retry 1–3 times with backoff
3. Still failing → Circuit Breaker opens
4. Next calls → instant Fallback response

The Bulkhead Pattern

Named after ship compartments — isolate failures so one problem can't sink the whole vessel.

Without Bulkhead

ONE shared thread pool
for ALL services

SlowService eats ALL 100 threads
→ Fast services also starve
→ Everything goes down

With Bulkhead

Payment
10 threads

Search
30 threads

Notif.
10 threads
✶ FULL

NotificationService hits its limit
→ Only IT is affected
→ Payment & Search keep running ✓

Each service gets its own thread pool

Concurrency limits per downstream
dependency

One slow service can't drain the shared
pool

Implementation with Polly

Polly is the .NET resilience library — add all patterns in a fluent, readable pipeline.

Install: `dotnet add package Microsoft.Extensions.Http.Resilience`

```
services
    .AddHttpClient<PaymentClient>()
    .AddResilienceHandler("payment", builder =>
    {
        builder
            .AddRetry(new HttpRetryStrategyOptions {
                MaxRetryAttempts = 3,
                Delay = TimeSpan.FromMilliseconds(200),
            })
            .AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions {
                FailureRatio = 0.5, // 50% failures trips it
                SamplingDuration = TimeSpan.FromSeconds(10),
                BreakDuration = TimeSpan.FromSeconds(30),
            })
            .AddTimeout(TimeSpan.FromSeconds(5));
    });
```

AddRetry()

Retry up to 3 times with 200ms delay between attempts. Good for transient network errors.

AddCircuitBreaker()

Open circuit if 50% of calls fail in 10s window. Stay open for 30s before testing recovery.

AddTimeout()

Abandon any call taking longer than 5 seconds. Frees threads immediately.

💡 Order matters: Retry wraps CircuitBreaker wraps Timeout — the outermost policy fires first

Every Language Has Its Polly

The pattern is identical everywhere — same 3 states. Only the syntax changes.

.NET**Polly**

YOU ARE HERE

Industry standard. Retry, CB, Timeout, Bulkhead, Fallback, Hedging. Part of .NET Foundation.

Java**Resilience4j**

Spring Boot

Replaced Netflix Hystrix. Annotations + YAML config. Deep Spring Boot integration.

Python**pybreaker**

pip install

Lightweight decorator-based library. Works with Flask, FastAPI, Django.

Node.js**Opossum**

npm install

Most popular Node.js CB library. v9 (2025) supports Node 20+. Also: Cockatiel (.NET Polly-inspired).

Kubernetes**Istio**

Infrastructure

Circuit breaker at the network/mesh level. No code changes needed. Configured via YAML.

Anti-Patterns to Avoid

✗ Retrying on Every Error

Don't retry 400 Bad Request — the request is wrong, retrying wastes resources. Only retry 5xx and timeouts.

✗ Sharing One Breaker

One circuit breaker instance shared across services = they all open together. Each downstream service needs its OWN instance.

✗ Threshold Too Sensitive

If the breaker opens after 1 failure, you'll have false positives constantly. Tune based on real traffic.

✗ Ignoring Half-Open State

Don't let too many test calls through in Half-Open. A flood of probes can re-break a recovering service.

✗ No Fallback

Opening the circuit without a fallback just changes the error message. Always define what to return when the circuit is open.

✗ No Monitoring

Circuit breakers are useless without observability. Track state changes, open durations, and fallback rates in your dashboard.

Key Takeaways

01

Cascading Failures are Real

One broken service can take down the entire platform without protection.

02

Circuit Breaker = 3 States

Closed (normal) → Open (tripped) → Half-Open (testing). It's a state machine.

03

Retry + CB + Fallback Together

Retry transient failures, break on sustained failures, always have a fallback.

04

Bulkhead Isolates Blast Radius

Give each service its own thread pool so failures stay contained.

05

Polly for .NET

AddRetry → AddCircuitBreaker → AddTimeout. Order matters. Chain them as a pipeline.